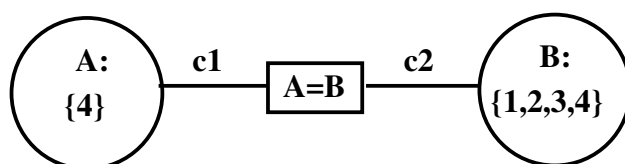

University of Waterloo
School of Computer Science
CS 486/686, **SAMPLE** Final Examination
Winter 2025

- Instructor: Jesse Hoey and Victor Zhong
- Date: Sometime Before April 22, 9:00am
- Location: Wherever
- Time: Whenever
- There are 5 questions on this exam
- There are 6 pages in this exam
- There are a total of 100 marks on the exam
- You have 150 minutes to complete the exam
- Non-programmable calculators are allowed
- Write your answers on a separate sheet On the REAL final, you will write directly on the exam (space will be provided)

Question 1: Short Answers (10 points out of a total of 100 points)

- (1a) [**2 points**] An *off-policy* reinforcement learning algorithm learns the value of the policy it is following while learning: state if this is true or false and write the name of an off-policy algorithm.
- (1b) [**2 points**] Cross-Validation is a common method to avoid _____
- (1c) [**2 points**] What is the difference between partially and fully observable domains?
- (1d) [**2 points**] What is the main reason that Heuristic Depth-First Search is often used in practice?
- (1e) [**2 points**] True or False: A single run of the AC-3 algorithm (without any domain splitting) is sufficient to decide if a constraint network has no solution.

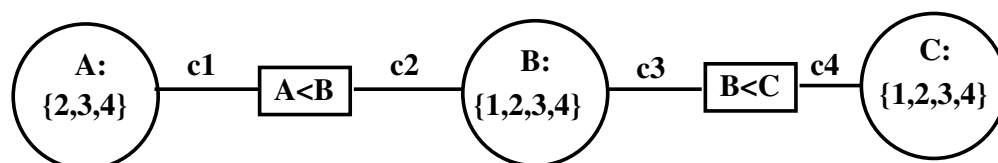
Question 2: CSP (15 points out of a total of 100 points)



Consider the simple constraint satisfaction problem shown above. The constraints are labeled c1,c2 and c1 is a constraint on A (by B), c2 is a constraint on B (by A). The table below shows the application of AC-3 to this CSP to make it arc-consistent. Each step is on a single line in the table, and a check-mark is under each constraint that is consistent (i.e. the arc is not in the TDA queue), and the domain values left for each variable are shown under each variable name. The next constraint to be made consistent is always chosen as the left-most one that is inconsistent.

A	c1	c2	B
4			1,2,3,4
4	✓		1,2,3,4
4		✓	4
4	✓	✓	4

Now consider this more complicated CSP:



[15 points] Complete the table below showing the application of AC-3 to this network. Do this in the same way as the example above, with one row per iteration. Always choose the left-most inconsistent constraint to make consistent next. Continue until the network is arc consistent. You don't have to re-copy the first two rows that are filled in below, but copy the headers for each column (i.e. put |A|c1|c2|B|c3|c4|C| as the first row of your table).

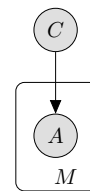
A	c1	c2	B	c3	c4	C
2,3,4			1,2,3,4			1,2,3,4
2,3	✓		1,2,3,4			1,2,3,4
... fill in the rest of this table ...						

Question 3: Bayesian Networks (30 points out of a total of 100 points)

Suppose you see your friend Bob jump off a cliff (J). Bob may have done so either because he is a mountain climber (C) and likes doing this (don't worry, Bob is safe if he is a climber since he will have a rope attaching him to the cliff, which you can't see), or because a bear is chasing him (B). Your prior belief that Bob is a climber is 0.05. Your prior belief that a bear is chasing Bob is 0.1. If a bear is actually chasing Bob, you believe Bob will jump with probability 0.75 if he is not a climber and with probability 0.99 if he is. If a bear is not chasing Bob, then you believe Bob will jump with probability 0.8 if he is a climber and 0.01 if he is not.

- (3a) [10 points] What is the probability that a bear is chasing Bob given you see Bob jump off the cliff (given that $J = \text{true}$). That is, what is $P(B|J = \text{true})$?
- (3b) [10 points] Suppose instead that you have three friends, Bob (same Bob as in last question), Bobby, and Roberta. Your beliefs about Bobby and Roberta are **the same** as those about Bob. That is, your prior belief that Bobby is a climber is 0.05 (and same for Bob and Roberta). If a bear is actually on the cliff chasing them, you believe Bobby will jump with probability 0.75 if he is not a climber and with probability 0.99 if he is (same for Bob and Roberta). If a bear is not chasing them, then you believe Bobby will jump with probability 0.8 if he is a climber and 0.01 if he is not (same for Bob and Roberta). That is, you have three friends, all of whom are identical to Bob. Use C_i to represent the fact that friend i is a climber and use J_i to represent the fact that friend i jumped off the cliff, where $i = \{1, 2, 3\}$ for Bob, Bobby and Roberta, respectively (although the order doesn't matter since they are all identical).

Draw the Bayesian network for this situation. Recall that “plate notation” allows you to represent a set of variables in a Bayesian network in a compact way. For example, a naïve Bayes model with class C and M features $A_1, A_2, \dots, A_M$ can be represented as shown on the right. Identical probability tables do not need to be replicated, so one only writes down two probability tables in this case.



- (3c) [10 points] [OPTIONAL QUESTION YOU CAN DO THIS FOR EXTRA PRACTICE OR GO STRAIGHT TO (3d)] Based on the problem stated in the previous question (3b), what is the probability that a bear is chasing your friends, given that you see Bob and Bobby jump off the cliff but you did not see if Roberta did or did not jump off the cliff because she was hidden behind a tree.
- (3d) [10 points] Again, based on the problem stated in question (3b), what is the probability that a bear is chasing your friends, given that you see Bob and Bobby jump off the cliff, but you also see Roberta not jump off the cliff.

Question 4: Decision Networks (25 points out of a total of 100 points)

RednitTM is a dating application (app) that is popular in Olertawo. You are being asked to construct a decision network to recommend users to other users. Users of the app can be one of two Types ($T \in \{A, B\}$), however this information is not available to the app for privacy reasons. The app does know, however, that 70% of users are type A (30% are type B). The app can also *Query* the user for their type, and the user's answer will be A or B , but possibly not be their true type (they may lie). Users of Type A will reveal their type honestly with probability 0.9, whereas for users of type B it is only 0.8. If they do not reveal their type honestly, they write opposite type (users of type A write their type as B 10% of the time, but users of type B write their type as A 20% of the time). The app's utility function is: it gets a reward of +10 if it recommends the correct match for a given type (i.e. it gets +10 if it recommends A to users whose actual type is A , regardless of what they replied to a query, and same for B) and 2 if it recommends the incorrect type (as this sometimes does work). Querying the user costs 4. If the app doesn't query the user, then the probability of the user's response is 0.5 for each type (i.e. the app doesn't know their response).

- (4a) [6 points] Draw a decision network to represent this problem. Show all conditional probability tables, and show the utility function.
- (4b) [15 points] Construct the optimal decision functions for both decisions and the optimal value for the entire network. Do this by first eliminating any variables that have no decision nodes as children. Each decision function should be a table showing what the app should do in each situation (i.e. in each assignment of values to variables in the information set of the decision). The final required optimal value should be a single number.
- (4c) [4 points] The app would benefit from knowing the users actual type, T , that is, the expected value would be higher if T was known. How much cost (in utility) would the app be willing to pay to find out the actual value of T ?

Question 5: Reinforcement Learning (20 points out of a total of 100 points)

A game-playing agent moves in a map with 5 locations: $(|l_1 \leftrightarrow l_2 \leftrightarrow l_3 \leftrightarrow l_4 \leftrightarrow g|)$, with $\leftrightarrow$ denoting that the agent can move between the locations and $|$ indicating a wall. The agent can either *move left* or *move right* or *pickup gold*. The agent gets a reward of +10 if it performs the action *pickup gold* at location g , and -1 if it hits a wall (i.e. moves left from l_1 or right from g). The agent's *move* actions are successful 80% of the time (with probability 0.2 the agent stays in the same place). The agent's *pickup gold* action resets the agent to location l_1 with certainty, regardless of which state it is performed in. Note that the agent cannot choose to stay in the same spot, it must try to move either left or right. The Q-value for the agent is initially as follows

Q(s,a) action	state				
	l_1	l_2	l_3	l_4	g
move right	8	13	20	17.5	17
move left	0	0	10	11	16
pickup gold	6	12.5	14	16.5	25

Using a discount factor of $\gamma = 0.9$ and a learning rate of $\alpha = 0.4$, use Q-learning to update the $Q(s, a)$ function for 4 iterations, given the agent starts in state l_1 , and acts greedily with respect to its current Q values, but that the second move action it takes fails (and the agent stays stationary). Assume the other three actions succeed. Show clearly the state s , the action taken a , the reward obtained, r , the new state s' , and the new $Q(s, a)$ value computed after each iteration. At the end, show the resulting Q function in a table like the one above. Show your work for updating each time.

Recall that a Q-learning agent updates $Q(s, a)$ after each experience tuple $\{s, a, r, s'\}$ as a weighted sum of its current estimate and the estimate from the experience given by $r + \gamma \max_{a'} Q(s', a')$, with weights given by $(1 - \alpha)$ and α , respectively.